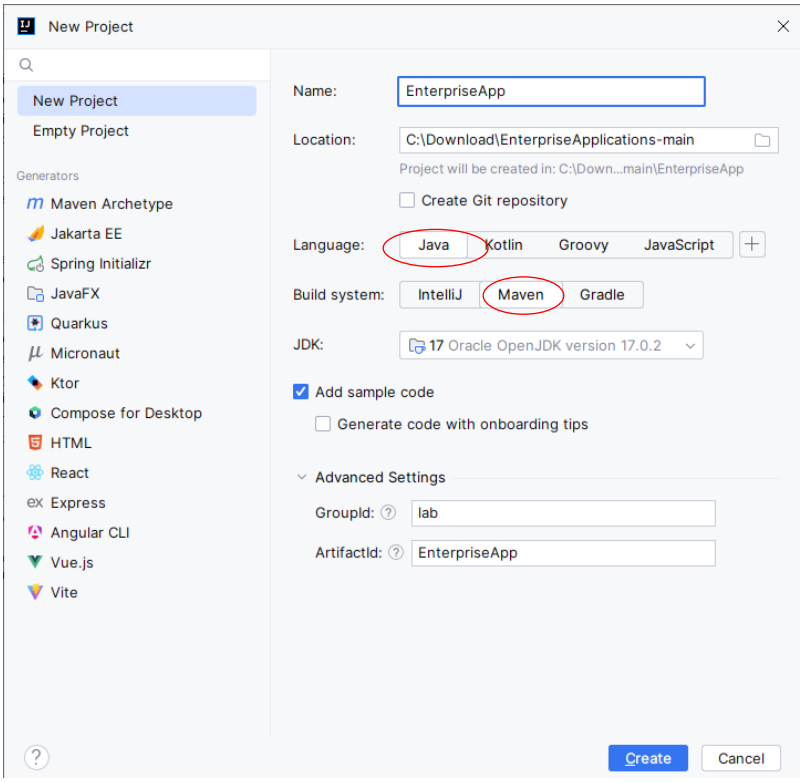
Message-Driven Beans

Celem ćwiczenia jest zapoznanie z komunikatowymi komponentami EJB w kontekście aplikacji Jakarta EE wykorzystującej również sesyjne EJB, encje JPA i strony JSF. Do realizacji ćwiczenia wymagane jest środowisko IntelliJ IDEA Ultimate wraz z serwerem aplikacji Payara i serwerem bazy danych H2 (dostępnym razem z serwerem aplikacji Payara). Oprócz podstawowych wtyczek obsługujących platformę Jakarta w IntelliJ IDEA Ultimate, wykorzystana będzie wtyczka „JPA Buddy". Jeśli nie jest jeszcze zainstalowana, to ją doinstaluj.

1. Uruchom IntelliJ IDEA i utwórz nowy projekt opcją New Project. W kreatorze projektu jako nazwę projektu podaj „EnterpriseApp". Nie wybieraj żadnego z generatorów (niech pozostanie zaznaczona opcja New Project). Wskaż język Java jako język programowania, a Maven jako system budowania aplikacji (Uwaga: domyślnie będzie zaznaczony IntelliJ). Jako wersję JDK wybierz 17 lub 11 jeśli 17 nie jest dostępna. W polu GroupId wpisz „lab", a w polu ArtifactId pozostaw treść zaproponowaną przez kreator, zgodną z nazwą projektu. Kliknij przycisk Create.

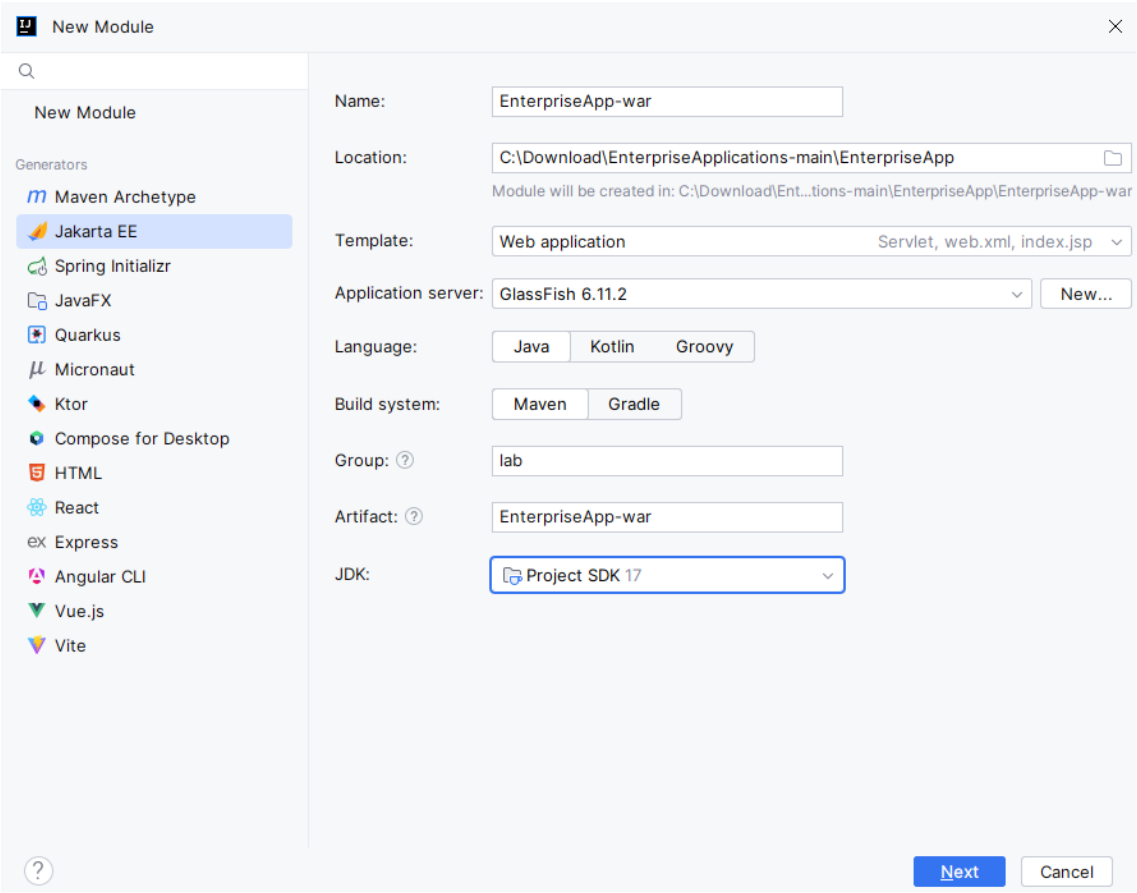


2. Usuń z projektu cały katalog `src`. Ten projekt będzie projektem macierzystym dla modułów składowych aplikacji. Nie będzie zawierał własnego kodu źródłowego.

3. Otwórz do edycji plik `pom.xml`. Ponieważ utworzony projekt nie będzie zawierał kodu źródłowe możemy usunąć sekcję <properties> z parametrami dla kompilatora. W jej miejsce wstaw poniższy element definiujący projekt jako macierzysty projekt składający się z modułów.

```
    <packaging>pom</packaging>
```

4. Utwórz w projekcie nowy moduł (New → Module z poziomu węzła projektu nadrzędnego). Jako nazwę modułu podaj „EnterpriseApp-war”, zaznacz w panelu generatorów Jakarta EE, a następnie z listy szablonów wybierz Web Application. Upewnij się, że jako serwer aplikacji wybrany jest skonfigurowany wcześniej serwer Payara (widoczny jako GlassFish), Java jako język programowania, a Maven jako system budowania aplikacji. W polu Group wpisz „lab”, a w polu Artifact pozostaw treść zaproponowaną przez kreator, zgodną z nazwą projektu. Jako wersję JDK pozostaw wersję powiązaną z projektem. Kliknij przycisk Next.



5. W drugim kroku kreatora projektu wybierz wersję platformy Jakarta Jakarta EE 10, a w panelu zależności zaznacz Full Platform. Kliknij przycisk Create.

6. Usuń z projektu utworzony przez kreator serwlet i stronę JSP.

   Uwaga: Gdyby gałąź modułu nie pojawiła się w drzewie projektu, skorzystaj z opcji Reload from Disk z poziomu węzła projektu.

7. Utwórz w projekcie drugi moduł (New → Module z poziomu węzła projektu nadrzędnego). Jako nazwę modułu podaj „EnterpriseApp-ejb”, zaznacz w panelu generatorów Jakarta EE, a następnie z listy szablonów wybierz Library. Upewnij się, że jako serwer aplikacji wybrany jest skonfigurowany wcześniej serwer Payara (widoczny jako GlassFish), Java jako język programowania, a Maven jako system budowania aplikacji. W polu Group wpisz

„lab”, a w polu Artifact pozostaw treść zaproponowaną przez kreator, zgodną z nazwą modułu. Jako wersję JDK pozostaw wersję powiązaną z projektem. Kliknij przycisk Next.

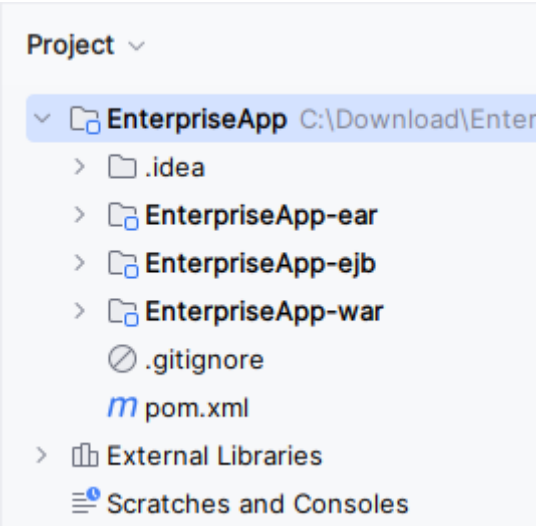
8. W drugim kroku kreatora projektu wybierz wersję platformy Jakarta Jakarta EE 10, a w panelu zależności zaznacz Full Platform. Kliknij przycisk Create.

9. Utwórz w projekcie trzeci moduł (New → Module z poziomu węzła projektu nadrzędnego). Jako nazwę modułu podaj „EnterpriseApp-ear”, zaznacz w panelu generatorów Jakarta EE, a następnie z listy szablonów wybierz Library. Upewnij się, że jako serwer aplikacji wybrany jest skonfigurowany wcześniej serwer Payara (widoczny jako GlassFish), Java jako język programowania, a Maven jako system budowania aplikacji. W polu Group wpisz „lab”, a w polu Artifact pozostaw treść zaproponowaną przez kreator, zgodną z nazwą modułu. Jako wersję JDK pozostaw wersję powiązaną z projektem. Kliknij przycisk Next.

10. W drugim kroku kreatora projektu wybierz wersję platformy Jakarta Jakarta EE 10, a w panelu zależności zaznacz Full Platform. Kliknij przycisk Create.

11. Usuń z modułu EAR cały katalog `src`. Ten moduł służy do spakowania archiwów pozostałych modułów w archiwum wielomodułowej aplikacji (EAR). Nie będzie zawierał własnego kodu źródłowego.

12. Upewnij się, że struktura projektu wygląda jak na poniższym obrazku. Jeśli niektóre moduły mimo ich utworzenia nie są widoczne, przeładuj nadrzędny projekt z dysku.



Rola poszczególnych węzłów projektu jest następująca:

a) Projekt nadrzędny, z poziomu którego można będzie inicjować budowanie całej aplikacji.
b) Moduł „ear” służący do utworzenia archiwum EAR łączącego moduły składowe aplikacji w archiwum typu Enterprise Archive.
c) Moduł „ejb” z komponentami Enterprise Beans (moduł EJB).
d) Moduł „war” z warstwą interfejsu użytkownika (moduł webowy).

13. Niestety konieczne jest ręczne poprawienie zawartości plików `pom.xml` projektu nadrzędnego i modułów składowych. Wykonaj kolejno poniższe kroki:

a) W pliku `pom.xml` projektu nadrzędnego po elemencie <packaging> dodaj poniższe informacje o modułach składowych:

```
<modules>
    <module>EnterpriseApp-war</module>
    <module>EnterpriseApp-ejb</module>
    <module>EnterpriseApp-ear</module>
</modules>
```

b) Pod koniec elementu <project> w plikach `pom.xml` wszystkich trzech modułów składowych wstaw na razie pusty element <parent>:

```
<parent>
</parent>
```

c) Wróć do pliku `pom.xml` projektu nadrzędnego i skopiuj do schowka elementy <groupId>, <artifactId> i <version>. Wklej je jako zawartość elementu <parent> w plikach `pom.xml` wszystkich trzech modułów składowych.

d) W sekcji <dependencies> pliku `pom.xml` modułu webowego dodaj jako zależność moduł EJB naszej aplikacji (zwróć uwagę by wartości elementów odpowiadały tym z pliku `pom.xml` modułu EJB):

```
    <dependency>
        <groupId>com.mycompany</groupId>
        <artifactId>EnterpriseApp-ejb</artifactId>
        <version>1.0-SNAPSHOT</version>
        <type>ejb</type>
        <scope>provided</scope>
    </dependency>
```

e) W sekcji <dependencies> pliku `pom.xml` modułu EAR dodaj jako zależności moduły webowy i EJB naszej aplikacji (zwróć uwagę by wartości elementów odpowiadały tym z plików `pom.xml` modułów):

```
<dependency>
    <groupId>lab</groupId>
    <artifactId>EnterpriseApp-ejb</artifactId>
    <version>1.0-SNAPSHOT</version>
    <type>ejb</type>
</dependency>
<dependency>
    <groupId>lab</groupId>
    <artifactId>EnterpriseApp-war</artifactId>
    <version>1.0-SNAPSHOT</version>
    <type>war</type>
</dependency>
```

Komentarz: Tym razem nie ustawiamy opcji <scope>provided</scope>, gdyż chcemy aby archiwa wskazanych modułów zostały zagnieżdżone w archiwum projektu EAR.

f) W sekcji <plugins> pliku `pom.xml` modułu EJB dodaj następującą wtyczkę:

```
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-ejb-plugin</artifactId>
            <version>3.2.1</version>
        </plugin>
```

g) W sekcji <plugins> pliku `pom.xml` modułu EAR dodaj następującą wtyczkę:

```
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-ear-plugin</artifactId>
            <version>3.3.0</version>
        </plugin>
```

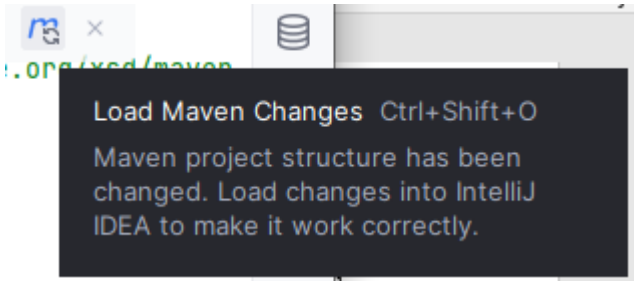
h) W pliku `pom.xml` modułu EJB po elemencie <name> dodaj element:

```
        <packaging>ejb</packaging>
```

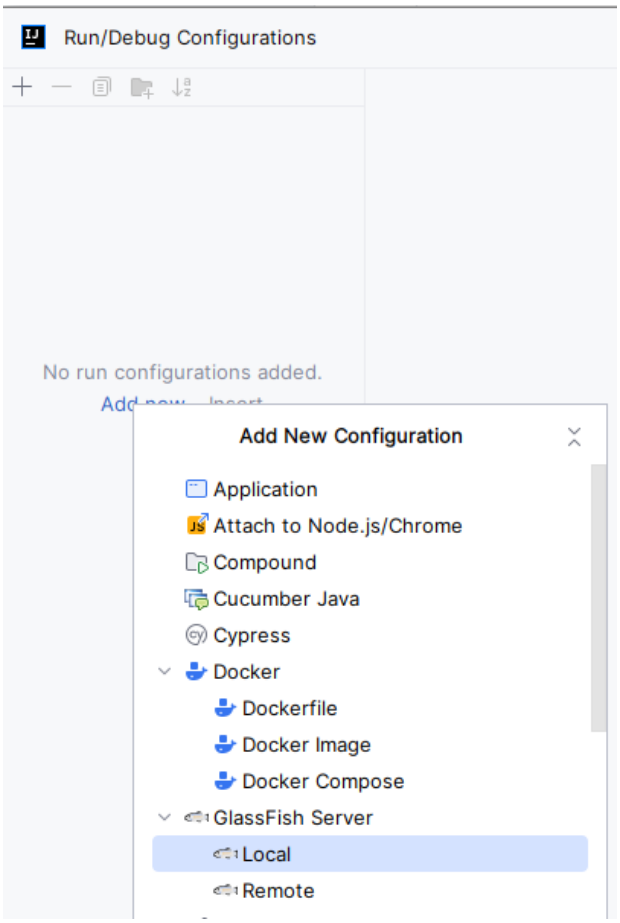
i) W pliku `pom.xml` modułu EAR po elemencie <name> dodaj element:

```
        <packaging>ear</packaging>
```

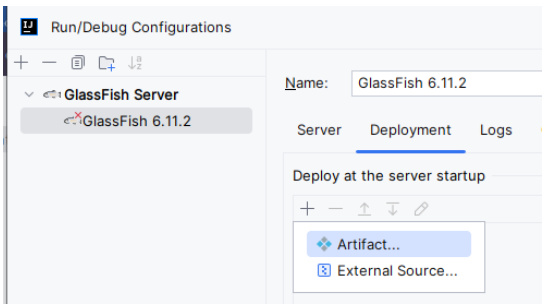
14. Zapisz wszystkie zmiany. Przejdź do edytora pliku `pom.xml` projektu nadrzędnego i przeładuj projekt Mavena np. klikając ikonę w edytorze.



15. Dodaj nową konfigurację uruchamiana aplikacji opcją Edit Configurations z poziomu paska narzędziowego. W wywołanym edytorze konfiguracji kliknij Add new..., a następnie wybierz z dostępnych opcji GlassFish Server/Local.



Na karcie Server edytora konfiguracji wybierz domenę, a następnie przełącz się na kartę Deployment. Dodaj nowy artefakt do instalowania na serwerze klikając ikonę plusa. Z listy dostępnych artefaktów wybierz „EnterpriseApp-ear:ear”.



16. W projekcie modułu EJB znajduje się już plik `persistence.xml`. Odszukaj go i otwórz do edycji w trybie tekstowym. Umieść w nim poniższą definicję jednostki trwałości (Persistence Unit), zastępując istniejącą, jeśli plik jakąś już zawiera:

```
<persistence-unit name="EnterpriseApp-ejbPU"
                  transaction-type="JTA">
 <jta-data-source>jdbc/__default</jta-data-source>
 <exclude-unlisted-classes>false</exclude-unlisted-classes>
 <properties>
 <property name="jakarta.persistence.schema-generation.database.action"
           value="create"/>
 </properties>
</persistence-unit>
```

17. W projekcie modułu EJB utwórz nową klasę encji do reprezentowania wiadomości prasowych posiadających nagłówek i treść:

    a) Kliknij prawym przyciskiem myszy na węźle `src/main/java` i z menu kontekstowego wybierz New → JPA Entity.

    b) Jako nazwę klasy podaj `NewsItem`, a jako nazwę pakietu `lab.ejb`. Wybierz z listy typ `Long` jako typ identyfikatora encji i zaznacz Auto jako sposób generowania wartości identyfikatorów (strategia dobrana do możliwości wykorzystywanego serwera bazy danych).

    c) Kliknij OK. Otwórz klasę encji do edycji i obejrzyj wygenerowany kod.

    d) Dodaj w klasie encji dwa prywatne pola typu `String` o nazwach `heading` i `body`. Wygeneruj dla nich publiczne metody setter/getter (wykorzystaj do tego celu kreator Refactor → Encapsulate Fields).

18. W projekcie modułu EJB utwórz komunikatowy komponent EJB, którego zadaniem będzie utrwalanie w bazie danych nadesłanego obiektu klasy `NewsItem`:

    a) Kliknij prawym przyciskiem myszy na węźle `src/main/java` i z menu kontekstowego wybierz New → Java Class. Jako nazwę klasy komponentu podaj `lab.ejb.NewsMDB`.

    b) Spraw dopisując klauzulę `implements` aby klasa implementowała interfejs `jakarta.jms.MessageListener`.

    c) Korzystając z menu kontekstowego edytora Generate → Implement Methods dodaj w klasie implementację metody zawartej w interfejsie `MessageListener`.

    d) Oznacz klasę adnotacją `@MessageDriven`, z atrybutami wskazującymi kolejkę, z której komponent będzie konsumował wiadomości:

```
@MessageDriven(activationConfig = {
    @ActivationConfigProperty(propertyName =
"destinationLookup", propertyValue = "java:app/jms/NewsQueue"),
    @ActivationConfigProperty(propertyName = "destinationType",
propertyValue = "jakarta.jms.Queue")
})
```

    Komponent może współpracować z zasobami JMS skonfigurowanymi przez administratora na serwerze aplikacji lub skonfigurowanymi w aplikacji, które zostaną dla niej utworzone w momencie jej instalacji na serwerze. Prefiks „java:app” w nazwie JNDI kolejki oznacza, że wykorzystamy drugie z możliwych rozwiązań.

    e) Ponieważ komponent będzie współpracował z kolejką skonfigurowaną w aplikacji należy taką konfigurację wyspecyfikować. Poprzedź adnotację `@MessageDriven` poniższą adnotacją `@JMSDestinationDefinition`:

```
@JMSDestinationDefinition(name = "java:app/jms/NewsQueue",
interfaceName = "jakarta.jms.Queue", resourceAdapter = "jmsra",
destinationName = "NewsQueue")
```

f) Zaimportuj wykorzystane adnotacje.

19. Zmodyfikuj kod komponentu komunikatowego w taki sposób, aby reagował na wiadomość zawierającą obiekt `NewsItem`, zapisując go do bazy danych:

a) Wstrzyknij zarządcę encji JPA do komponentu komunikatowego wstawiając poniższy kod przed konstruktorem (pamiętaj o importach):

```
    @PersistenceContext
    private EntityManager em;
```

b) Zmodyfikuj metodę `onMessage` umieszczając w niej poniższy kod (zaimportuj również wykorzystywane klasy biblioteczne JMS):

```
        ObjectMessage msg = null;
         try {
             if (message instanceof ObjectMessage) {
                 msg = (ObjectMessage) message;
                 NewsItem e = (NewsItem) msg.getObject();
                 em.persist(e);
             }
         } catch (JMSException e) {
             e.printStackTrace();
         }
```

20. W module EJB utwórz sesyjną fasadę do obsługi encji z poziomu wyższych warstw aplikacji:

a) Kliknij prawym przyciskiem myszy na węźle `src/main/java` i z menu kontekstowego wybierz New → Java Class. Zaznacz, że chcesz utworzyć interfejs a nie klasę! Jako nazwę interfejsu podaj `lab.ejb. NewsItemFacadeLocal`.

b) Oznacz interfejs adnotacją `@Local`, aby pełnił rolę interfejsu lokalnego dla klasy komponentu EJB, która będzie go implementować.

Uwaga: Nie przejmuj się błędem zgłaszanym przez IntelliJ, sugerującym że użycie technologii EJB nie jest możliwe z powodu braku bibliotek. Jest to bug w IDE.

c) Kliknij prawym przyciskiem myszy na węźle `src/main/java` i z menu kontekstowego wybierz New → Java Class. Jako nazwę klasy komponentu podaj `lab.ejb. NewsItemFacade`.

d) Oznacz klasę adnotacją `@Stateless` czyniąc z niej klasę komponentu EJB i jednocześnie wskazując, że komponent ma być bezstanowy.

e) Spraw by klasa implementowała utworzony przed chwilą interfejs.

f) Wstrzyknij do klasy komponentu zarządcę encji JPA w ten sam sposób co wcześniej w komponencie komunikatowym.

g) Dodaj w komponencie sesyjnym poniższą metodę zwracającą wynik zapytania zbudowanego z wykorzystaniem Criteria API JPA, pobierającego z bazy danych wszystkie newsy. Zaimportuj wszystkie wykorzystywane klasy/interfejsy.

```
public List<NewsItem> getAllNewsItems() {
  CriteriaBuilder cb = em.getCriteriaBuilder();
  CriteriaQuery<NewsItem> cq = cb.createQuery(NewsItem.class);
  Root<NewsItem> rootEntry = cq.from(NewsItem.class);
  CriteriaQuery<NewsItem> ct = cq.select(rootEntry);
  TypedQuery<NewsItem> allNewsItemsQuery = em.createQuery(ct);
  return allNewsItemsQuery.getResultList();
}
```

h) Udostępnij metodę dodaną do komponentu sesyjnego przez jego interfejs lokalny, edytując ręcznie kod interfejsu.

21. W projekcie modułu webowego utwórz nową stronę JSF (New → JSF/Facelets z menu kontekstowego węzła `webapp`). Jako jej nazwę podaj `news`. Zwróć uwagę aby z dwóch dostępnych opcji tworzonego pliku wybrana była opcja JSF File.

22. Przejdź do edycji źródła strony JSF (`news.xhtml`). Zastąp opcjonalny element `<f:view>` (wraz z jego zawartością) poniższym kodem.

```
   <h:head>
       <title>News</title>
   </h:head>
   <h:body>
   </h:body>
```

23. Usuń z kodu strony deklarację przestrzeni nazw powiązaną z prefiksem `ui`.

24. Otwórz do edycji plik `web.xml`. W obrębie elementu `<web-app>` umieść poniższy kod konfigurujący kontroler JSF.

```
    <servlet>
        <servlet-name>Faces Servlet</servlet-name>
        <servlet-class>
           jakarta.faces.webapp.FacesServlet
        </servlet-class>
        <load-on-startup>1</load-on-startup>
    </servlet>
    <servlet-mapping>
        <servlet-name>Faces Servlet</servlet-name>
        <url-pattern>/faces/*</url-pattern>
    </servlet-mapping>
```

25. Utwórz w projekcie modułu webowego komponent CDI, który będzie pełnił funkcję backing bean dla utworzonej w poprzednim kroku strony JSF. Komponent będzie

udostępniał m.in. metody do odczytu wiadomości z bazy danych i do wysyłania wiadomości do kolejki JMS:

a) Utwórz w projekcie klasę `lab.backing.NewsBean`.

b) Oznacz klasę adnotacją ustawiającą typowy dla komponentów backing bean zasięg (`@RequestScoped`).

c) Oznacz klasę adnotacją `@Named`, dzięki której komponent CDI będzie dostępny z poziomu języka wyrażeń na stronie JSF.

d) Dodaj w klasie komponentu CDI (`NewsBean.java`) wstrzyknięcie referencji do komponentu fasadowego EJB poprzez jego interfejs lokalny. Wykorzystaj poniższy kod. Nie zapomnij o wymaganych importach.

```
    @Inject
    private NewsItemFacadeLocal facade;
```

e) Dodaj w klasie komponentu CDI podane poniżej wstrzyknięcie zależności dla kontekstu JMS oraz kolejki wiadomości (wykorzystana w kodzie adnotacja to `jakarta.annotation.Resource`):

```
    @Inject
    private JMSContext context;
    @Resource(lookup="java:app/jms/NewsQueue")
    private  jakarta.jms.Queue queue;
```

Uwaga: Powyższa adnotacja `@Resource` wskazuje kolejkę, która zostanie automatycznie skonfigurowana na serwerze przy instalacji aplikacji (tę samą, z której wiadomości pobierać ma utworzony wcześniej komunikatowy EJB).

f) Dodaj w klasie komponentu CDI poniższą metodę, wysyłającą wiadomość z obiektem `NewsItem` do kolejki (zaimportuj wykorzystywane klasy biblioteczne JMS i klasę encji):

```
void sendNewsItem(String heading, String body) {
  try {
    ObjectMessage message = context.createObjectMessage();
    NewsItem e = new NewsItem();
    e.setHeading(heading);
    e.setBody(body);
    message.setObject(e);

    context.createProducer().send(queue, message);
  } catch (JMSException ex) {
     ex.printStackTrace();
  }
}
```

g) Jeśli kompilator wskaże błąd w instrukcji ustawiającej instancję encji jako zawartość wiadomości, informując, że oczekiwany był obiekt typu `Serializable`, spraw aby klasa encji implementowała ten interfejs.

h) Dodaj w klasie komponentu CDI metodę (niekompletny kod metody poniżej) odczytującą listę wszystkich obiektów `NewsItem` z bazy danych poprzez wstrzyknięty komponent fasadowy:

```
public List<NewsItem> getNewsItems()
{
    return ...;
}
```

26. Umieść na stronie JSF formularz umożliwiający wysyłanie nowych wiadomości do kolejki. Formularz powinien zawierać następujące komponenty JSF: dwa pola tekstowe (znacznik `<h:inputText>`, identyfikatory pól: `headingInputText` i `bodyInputText`) do wprowadzania danych poprzedzone etykietami (znacznik `<h:outputText>`) oraz przycisk (znacznik `<h:commandButton>`, identyfikator: `submitButton`) uruchamiający akcję.

    Uwaga: Pamiętaj, że elementy formularza muszą być otoczone elementem `<h:form>`.

27. Dla obu pól formularza do wprowadzania danych utwórz odpowiadające im właściwości typu `String` w klasie komponentu CDI (o nazwach `headingText` i `bodyText`) i powiąż je z wartościami komponentów.

```
...
<h:inputText id="headingInputText"
             value="#{newsBean.headingText}"> </h:inputText>
...
<h:inputText id="bodyInputText"
             value="#{newsBean.bodyText}"></h:inputText>
...
```

28. Dodaj w kodzie komponentu CDI metodę akcji (niekompletny kod metody poniżej) i powiąż ją z przyciskiem formularza. Metoda akcji powinna wywoływać metodę `sendNewsItem`, przekazując jej jako parametry wartości pochodzące z pól formularza.

```
public String submitNews()
{
    ...;
    return null;
}
```

29. Na końcu formularza na stronie JSF umieść komponent tabelki do wyświetlania wiadomości zapisanych w bazie danych:

```
          <h:dataTable value="#{newsBean.newsItems}" var="item"
                       border="1">
                <h:column>
                    <f:facet name="header">
                        <h:outputText value="Id"/>
                    </f:facet>
                    <h:outputText value="#{item.id}"/>
```

```
                </h:column>
                <h:column>
                    <f:facet name="header">
                        <h:outputText value="Heading"/>
                    </f:facet>
                    <h:outputText value="#{item.heading}"/>
                </h:column>
                <h:column>
                    <f:facet name="header">
                        <h:outputText value="Body"/>
                    </f:facet>
                    <h:outputText value="#{item.body}"/>
                </h:column>
            </h:dataTable>
```

30. Przy zaznaczonym w panelu struktury projektu projekcie nadrzędnym uruchom konsolę IntelliJ dwukrotnie naciskając klawisz Ctrl. Z poziomu konsoli zbuduj aplikację komendą Mavena `mvn install` (IntelliJ nie radzi sobie z tak skomplikowaną strukturą projektu Mavena…). Odszukaj na dysku powstałe archiwum EAR i podejrzyj jego zawartość, zwracając uwagę na zagnieżdżone archiwa modułów składowych i wygenerowany przez wtyczkę Mavena plik konfiguracyjny `application.xml`.

31. Uruchom aplikację i przetestuj jej działanie. Popraw adres w przeglądarce na http://localhost:8080/EnterpriseApp-war/faces/news.xhtml. Wyślij kilka wiadomości. Spróbuj zaobserwować efekt komunikacji asynchronicznej.

    Komentarz: Ze względu na asynchroniczną realizację dodawania nowych informacji, może się zdarzyć, że zawartość tabelki na stronie będzie odświeżona zanim nowy wiersz trafi do bazy danych za pośrednictwem kolejki i komponentu komunikatowego.

Zadanie do samodzielnego wykonania

Zmodyfikuj aplikację tak, aby do kolejki była wysyłana wiadomość `TextMessage` (zamiast `ObjectMessage`). Treścią wiadomości powinien być tekst składający się z nagłówka i treści newsa, oddzielonych pionową kreską. Komponent komunikatowy powinien podzielić odebrany tekst na nagłówek i treść newsa, utworzyć obiekt encji `NewsItem` i utrwalić go w bazie danych.